

Robot Operating System (ROS)

Walter Fetter Lages

fetter@ece.ufrgs.br

Universidade Federal do Rio Grande do Sul

Escola de Engenharia

Departamento de Sistemas Elétricos de Automação e Energia

ENG10051 Dinâmica e Controle de Robôs



Introdução

ROS é um pseudo sistema operacional com ferramentas para desenvolvimento de *software* para robôs:

- Gerenciamento de pacotes
- Abstração de *hardware*
- Bibliotecas com algoritmos comumente utilizados
- Simuladores
- Mecanismos de comunicação
- *scripts* úteis

Porque ROS?

- Código aberto
- Centralização das informações
- Reuso de código
- Desenvolvimento em grupo
- Processamento inerentemente distribuído
- Nós fracamente acoplados
- Parar de reinventar a roda

Histórico

- Sistema desenvolvido em Stanford em 2000 para o robô STAIR 1
- Aperfeiçoado em 2007 pela Willow Garage para o robô PR2 e denominado ROS





Versões do ROS 1

- Noetic Ninjemys - 23 de maio de 2020
- Melodic Morenia - 23 de maio de 2018
- Lunar Loggerhead - 23 de maio de 2017
- Kinetic Kame - 23 de maio de 2016
- Jade Turtle - 23 de maio de 2015
- Indigo Igloo - 22 de julho de 2014
- Hydro Medusa - 4 de setembro de 2013
- Groovy Galapagos - 31 de dezembro de 2012
- Fuerte Turtle - 23 de abril de 2012
- Electric Emys - 30 de agosto de 2011
- Diamond Back - 2 de março de 2011
- C Turtle - 2 de agosto de 2010
- Box Turtle - 2 de março de 2010

Sistema Operacional *Host*

- Linux é o sistema operacional *host*
- A distribuição "oficial" é a Ubuntu
- Outras distribuição suportadas:
 - Ubuntu ARM
 - OS X
 - Yocto
 - Debian
 - Arch Linux
 - Ångström
 - UDOO
 - Android
 - Código fonte
 - robotpkg

Conceitos

- O ROS é organizado em pacotes
 - Nodos (processo do sistema operacional hospedeiro)
 - programas utilitários
 - Bibliotecas
 - Definições de mensagens
 - Arquivos de configuração
 - *Plugins*
- Pacotes podem ser agrupados em metapacotes



Pacotes

- Menor nível na organização do ROS
- Dedicados a uma única funcionalidade
- Cada pacote deve ser implementado em um diretório

Estrutura de um Pacote ROS 1

```
catkin_ws/  
├── build/  
│   ├── nome_do_pacote/ ..... Temporários  
│   └── devel/  
│       ├── lib/ ..... Bibliotecas  
│       ├── nome_do_pacote/ ..... Executáveis  
│       └── setup.bash ..... Script de configuração  
└── src/  
    ├── nome_do_pacote/  
    │   ├── src/ ..... Códigos-fonte  
    │   ├── launch/ ..... Scripts de carga  
    │   ├── scripts/ ..... Outros scripts  
    │   ├── include/ ..... Arquivos de cabeçalho  
    │   ├── package.xml ... Metadados e dependências  
    │   └── CMakeList.txt .. Configuração do Cmake
```

Metapacotes

- Agrupam pacotes que em conjunto oferecem uma funcionalidade mais abstrata
- Metapacotes são pacotes “vazios”
 - Servem para agrupar outros pacotes como dependências
 - Baixando um metapacote, se baixa todas as dependências
 - Pacotes não devem depender de metapacotes, apenas de pacotes

roscore

- `roscore` é um conjunto de nodos e programas que são pré-requisitos para um sistema ROS 1
- Deve ser executado para que os nodos do ROS 1 possam se comunicar
- É lançado com o comando `roscore`
- O comando `roslaunch` também lança o `roscore`, se ele ainda não estiver executando
- O `roscore` inicia os seguintes nodos:
 - ROS master
 - ROS parameter server
 - `rosout`

ROS Master

- Nodo que provê serviços de registro e consulta de nomes de nodos, tópicos e serviços
- Ao iniciar, os nodos devem registrar-se com o mestre
- Ao subscrever um tópico, os nodos consultam o mestre e estabelecem conexão diretamente entre si
- É lançado pelo comando `roscore`
- O computador onde o *ROS Master* está executando é especificado através da variável de ambiente `ROS_MASTER_URI`

Servidor de Parâmetros

- Nodo que provê um dicionário acessível aos demais nodos
- É lançado pelo comando `roscore`
- Nodos podem usar o servidor de parâmetros para armazenar e recuperar parâmetros
- Não projetado para alto desempenho
 - Adequado para parâmetros de configuração
 - Não é um substituto para aplicações de banco de dados

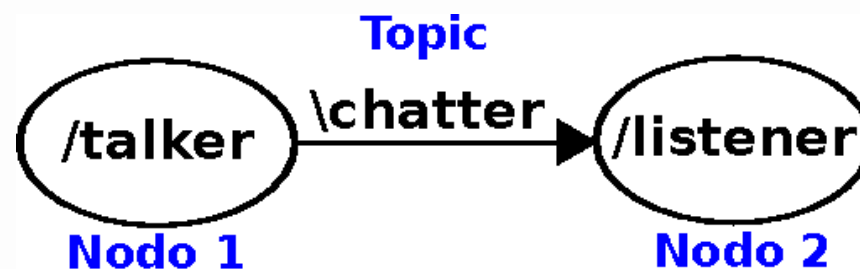
Comunicação através de Tópicos

Nodo: processo do S.O. hospedeiro

Tópico: mecanismo de comunicação entre nodos do tipo *publisher/subscriber*

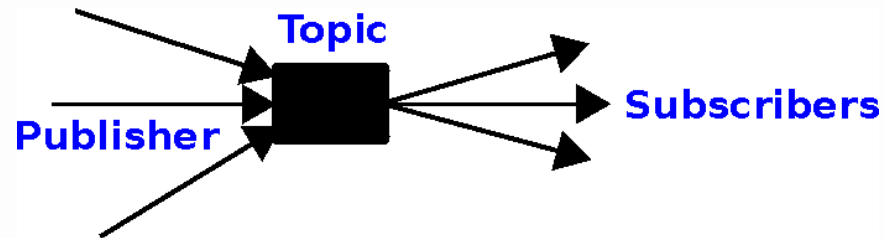
Mensagem: dados publicados nos tópicos

Gráfico de computação: Representa a comunicação entre os nodos através de tópicos



Tópicos

- Nós podem publicar mensagens em tópicos
- Cada tópico pode ter vários publicadores e assinantes

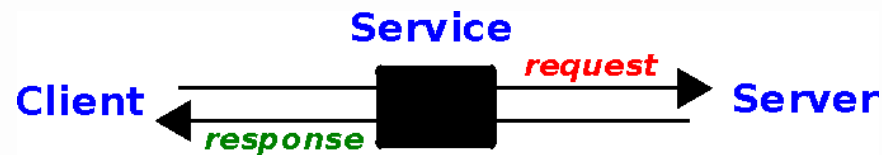


- Cada nó pode publicar ou assinar vários tópicos
- Publicadores e assinantes não sabem da existência um dos outros
- A ordem de execução não é garantida
- Comunicação assíncrona

Serviços

- Tópicos não são apropriados para solicitação de serviços entre nodos

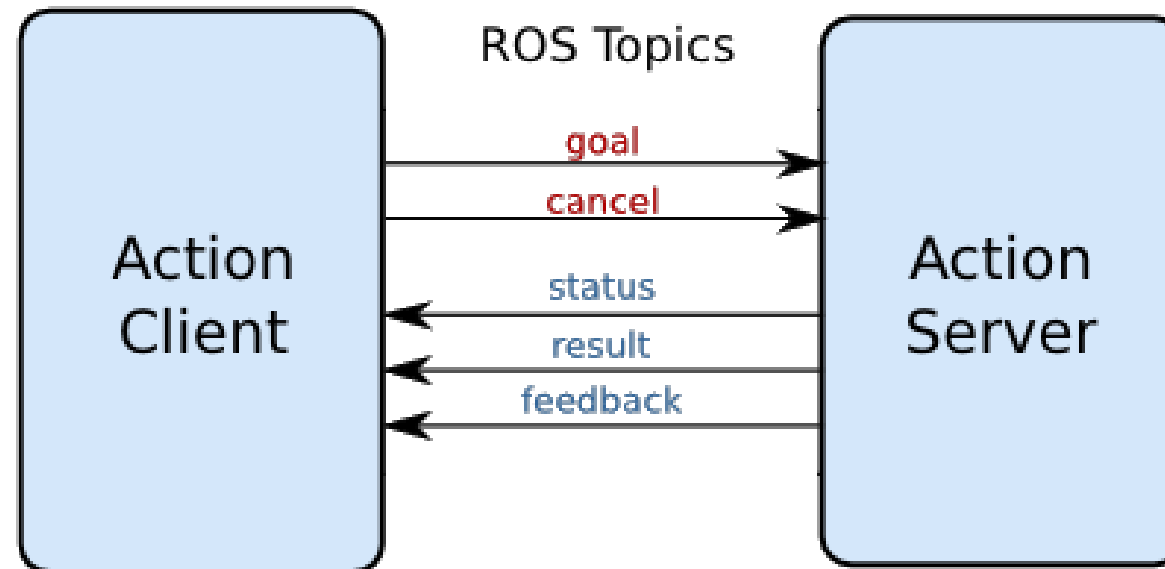
Serviço: mecanismo de comunicação entre nodos do tipo *remote procedure call* (RPC)



- Serviços oferecem um mecanismo de requisição/resposta

Action Servers

- Apropriados para serviços exigem longo tempo de execução
- Permitem o cancelamento da requisição
- Permitem receber informações sobre o *status* da execução



Rviz

- Rviz é uma ferramenta do ROS para visualização
- Mostra de forma conveniente dados publicados em tópicos

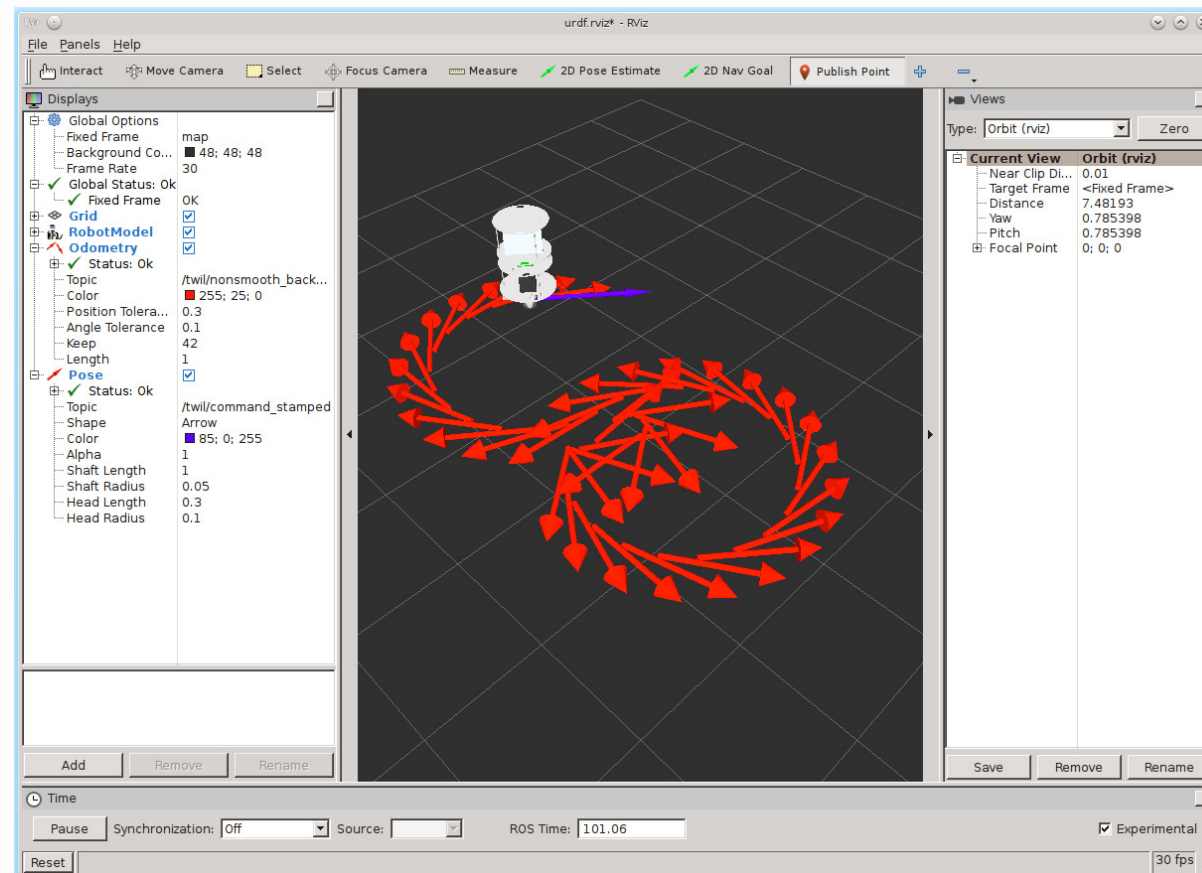
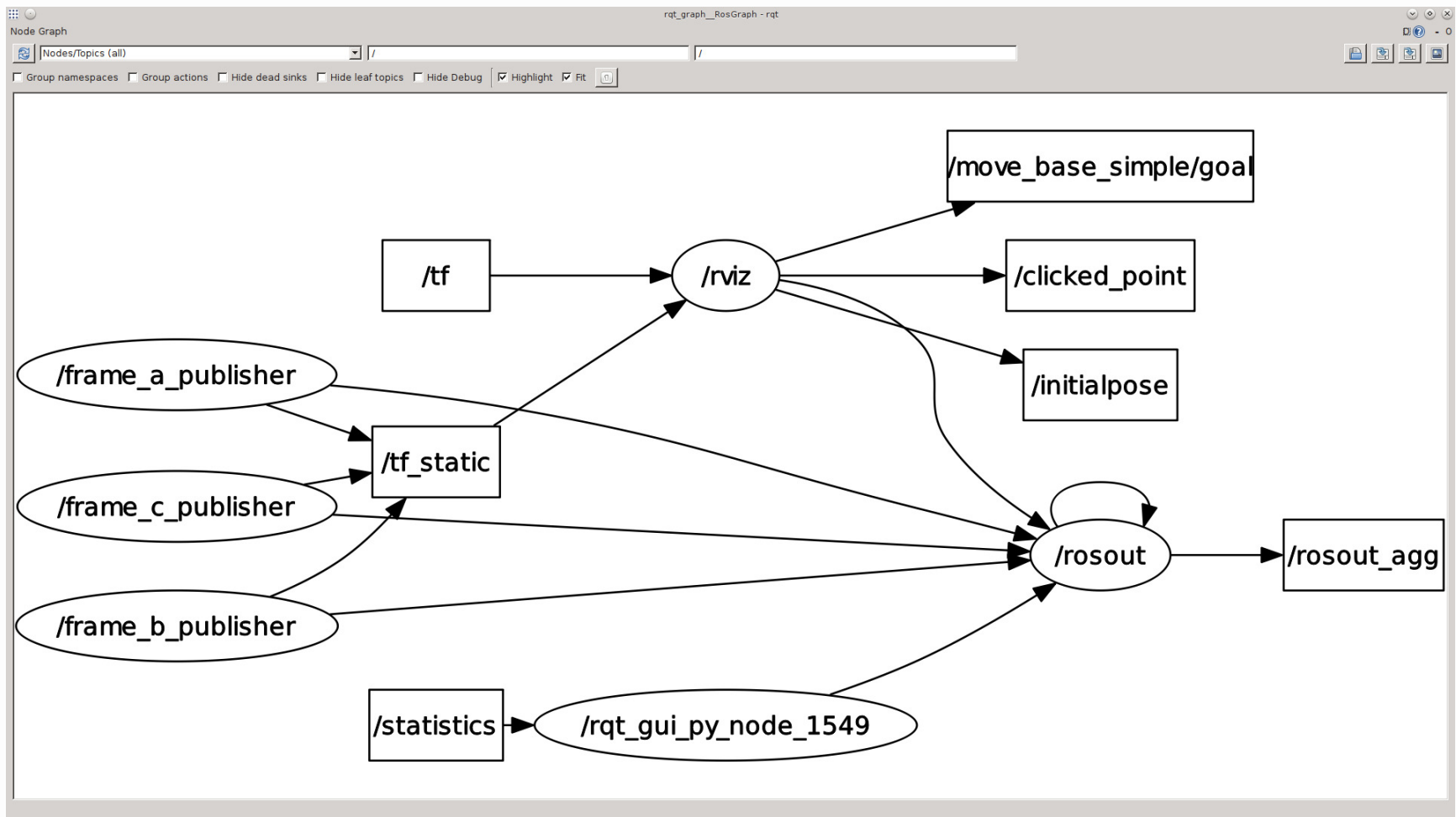


Gráfico de Computação

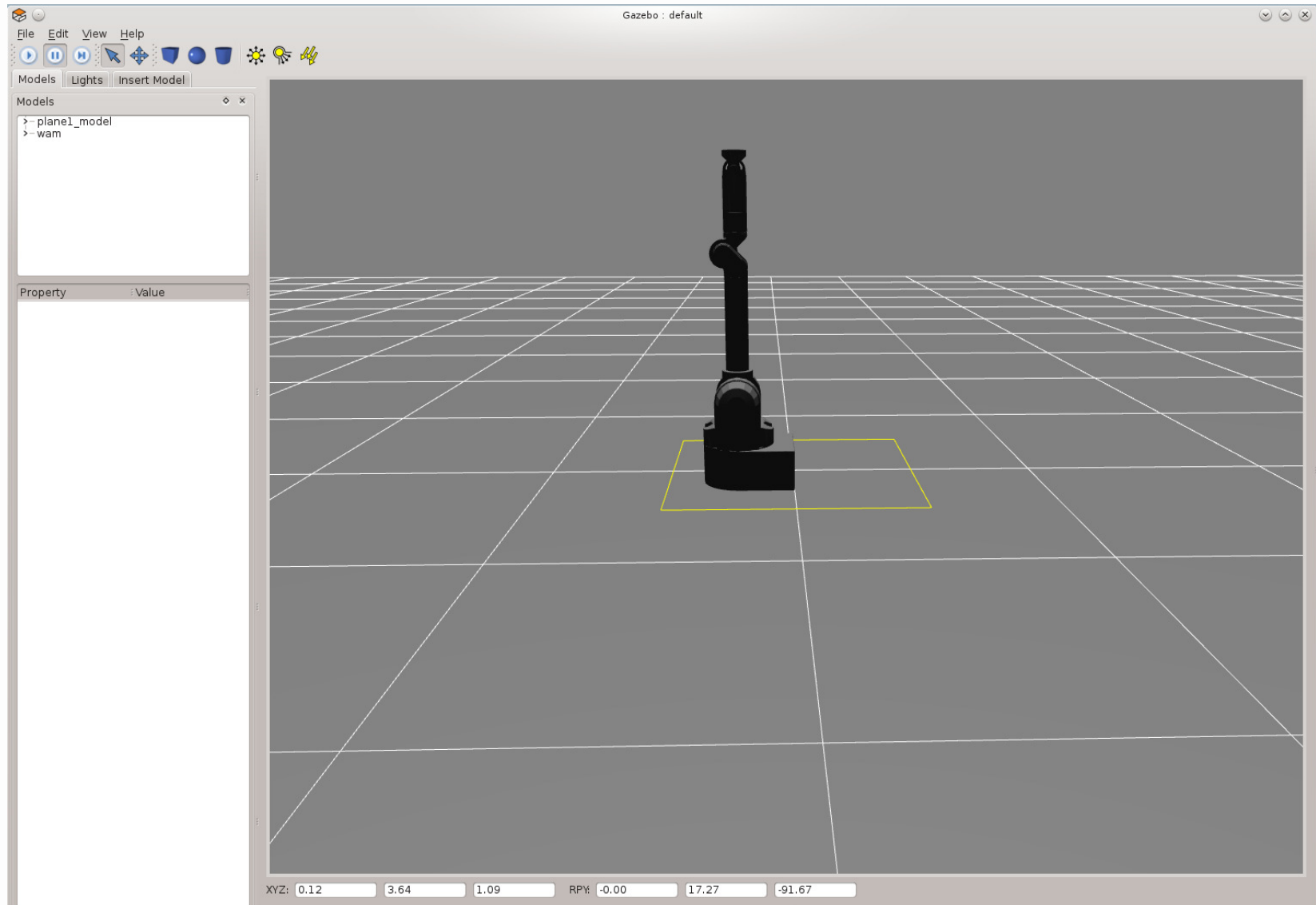
- A ferramenta `rqt_graph` Permite visualizar os nodos e tópicos



Gazebo

- Simulador 3D
- Suporta diversos *backends*
 - *Open Dynamics Engine* (ODE)
 - *Bullet*
- O modelo do robô é descrito em URDF

Barrett WAM no Gazebo



ROS2

- O ROS foi criado como um sistema de desenvolvimento para o PR2
 - Suporta apenas um robô
 - Recursos computacionais abundantes *on board*
 - Sem requisitos de tempo-real
 - Excelente conectividade
 - Aplicações em pesquisa
 - Máxima flexibilidade (sem *warp* do `main()`)

Porque ROS2?

- Novos casos de uso: times de robôs, plataformas embarcadas, sistemas de tempo-real, rede não ideal, ambientes de produção (ROS-I)
- Novas tecnologias: Zeroconf, Protocol buffers, ZeroMQ, Redis, WebSockets, Data Distribution Service (DDS)
 - Já é possível construir o *middleware* com componentes de prateleira
- Mudanças na API
 - Muitas mudanças poderiam instabilizar o ROS1

Diferenças do ROS2

- Testado nas plataformas Ubuntu, OS X e Windows 10
- Cada pacote tem a sua configuração de ambiente
- Apenas *isolated build* é suportado
- Sem diretório devel, pacotes precisam ser instalados para serem usados
- *Namespaces* separados para `.msg` e `.srv`
- Campos primitivos em mensagens podem ter valores *default*
- Tipos `duration` e `time` unificados
- Cabeçalhos de mensagens sem o campo `seq`

Diferenças do ROS2

- Não é preciso executar o `roscore`
- Nodos tem um ciclo de vida que pode ser gerenciado por ferramentas
- Suporte para tempo-real (se o sistema operacional suportar)
- Arquivos de *launch* escritos em Python, XML ou YAML
- Pacotes binários para o Windows baseados no Chocolatey

Versões do ROS2

- Rolling Ridley - desde junho 2020
 - Rolling release
- Iron Irwini - 23 de maio de 2023
- Humble Hawksbill - 23 de maio de 2022
- Galactic Geochelone - 23 de maio de 2021
- Foxy Fitzroy - 5 de junho de 2020
- Eloquent Elusor - 22 de novembro de 2019
- Dashing Diademata - 31 de maio de 2019
- Crystal Clemmys - 14 de dezembro de 2018
- Bouncy Bolson - 2 de julho de 2018
- Ardent Apalone - 8 de dezembro de 2017

Estrutura de um Pacote ROS 2

```
colcon_ws/  
├── build/  
│   ├── nome_do_pacote/ ..... Temporários  
│   └── install/  
│       ├── nome_do_pacote/  
│           ├── lib/ ..... Bibliotecas e executáveis  
│           ├── share/ ..... Configuração e launch  
│           ├── local_setup.bash .. Script de configuração local  
│           ├── setup.bash ..... Script de configuração global  
│           └── log/ ..... logs  
│       └── src/  
│           ├── nome_do_pacote/  
│               ├── src/ ..... Códigos-fonte  
│               ├── launch/ ..... Scripts de carga  
│               ├── scripts/ ..... Outros scripts  
│               ├── include/ ..... Arquivos de cabeçalho  
│               ├── package.xml ..... Metadados e dependências  
│               └── CMakeList.txt ..... Configuração do Cmake
```

Instalação

- Sistema operacional *host*
 - Linux
 - Ubuntu (Kubuntu)
 - Pacotes Debian
 - Arquivo *fat*
 - Red-Hat Enterprise Linux
 - Pacotes RPM
 - Arquivo *fat*
 - Windows 10
 - macOS
 - Instalação a partir do código-fonte
 - Deve funcionar em qualquer Linux

Instalação

- Instalação do Linux
 - Dual-boot
 - Máquina virtual
 - Problemas com algumas placas gráficas
 - Docker
- Instalação do ROS2: <http://docs.ros.org>
 - Versão Humble
 - Instalada a partir de pacotes Debian
 - Instalada em /opt/ros/humble